

CA2 - Supervised machine learning classification pipeline - applied to medical data

Important information

- Do **not** use scikit-learn (`sklearn`) or any other high-level machine learning library for this CA
- Explain your code and reasoning in markdown cells or code comments
- Label all graphs and charts if applicable
- If you use code from the internet, make sure to reference it and explain it in your own words
- If you use additional function arguments, make sure to explain them in your own words
- Use the classes `Perceptron`, `Adaline` and `Logistic Regression` from the library `mlxtend` as classifiers (from `mlxtend.classifier` import `Perceptron`, `Adaline`, `LogisticRegression`). *Always* use the argument `minibatches=1` when instantiating an `Adaline` or `LogisticRegression` object. This makes the model use the gradient descent algorithm for training. *Always* use the `random_seed=42` argument when instantiating the classifiers. This will make your results reproducible.
- You can use any plotting library you want (e.g. `matplotlib`, `seaborn`, `plotly`, etc.)
- Use explanatory variable names (e.g. `X_train` and `X_train_scaled` for the training data before and after scaling, respectively)
- The dataset is provided in the file `fetal_health.csv` in the `assets` folder

Additional clues

- Use the `pandas` library for initial data inspection and preprocessing
- Before training the classifiers, convert the data to raw `numpy` arrays
- For Part IV, you are aiming to create a plot that looks similar to this:

Additional information

- Feel free to create additional code or markdown cells if you think it will help you explain your reasoning or structure your code (you don't have to).

Part I: Data loading and data exploration

Import necessary libraries/modules:

```
# Insert your code below
# =====

import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
```

```
from mlxtend.classifier import Perceptron, Adaline, LogisticRegression
```

Loading and exploring data

1. Load the dataset `fetal_health.csv` with `pandas`. Use the first column as the row index.
2. Check for missing data, report on your finding and remove samples with missing data, if you find any.
3. Display the raw data with appropriate plots/outputs and inspect it. Describe the distributions of the values of feature "`baseline value`", "`accelerations`", and the target variable "`fetal_health`".
4. Will it be beneficial to scale the data? Why or why not?
5. Is the data linearly separable using a combination of any two pairs of features? Can we expect an accuracy close to 100% from a linear classifier?

```
# Insert your code below
# =====

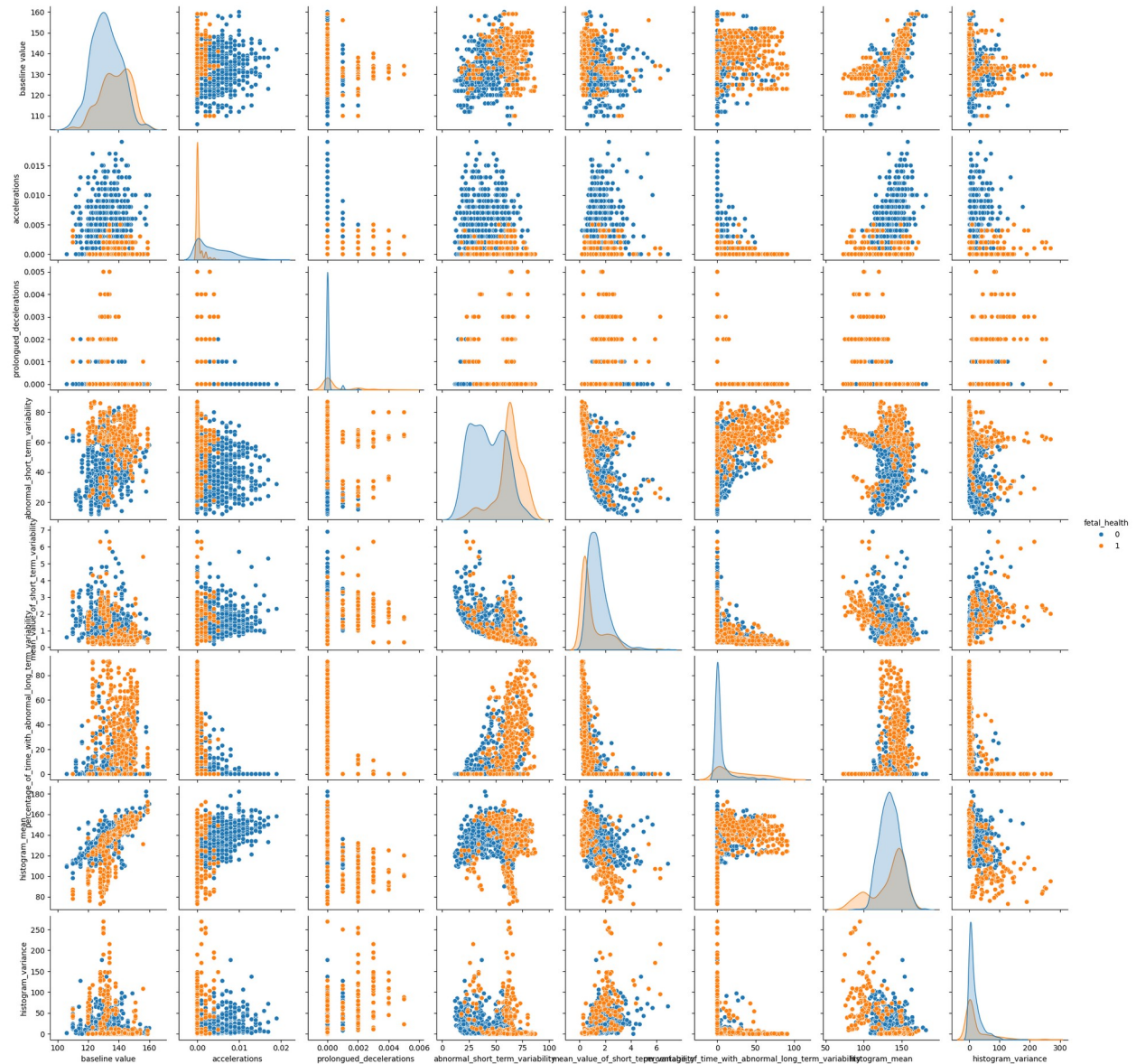
#Loading and indexing data
df = pd.read_csv('assets/fetal_health.csv', index_col=[0])

# Checking for missing data
nan_rows = df[df.isna().any(axis=1) == True]
print(f'Number of NaN rows: {len(nan_rows)}')

sns.pairplot(data=df, hue='fetal_health')

Number of NaN rows: 0

<seaborn.axisgrid.PairGrid at 0x111c03d30>
```



Distribution description of 'baseline value', 'accelerations' and 'fetal health'

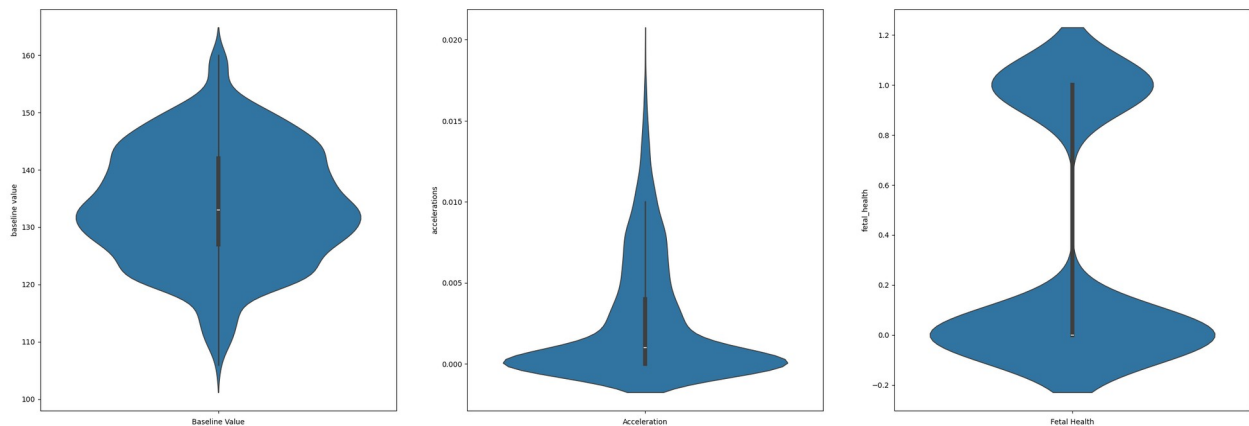
```
fig, ax = plt.subplots(1, 3, figsize=(30, 10))

sns.violinplot(data=df['baseline value'], ax = ax[0])
ax[0].set_xlabel('Baseline Value')

sns.violinplot(data=df['accelerations'], ax = ax[1])
ax[1].set_xlabel('Acceleration')

sns.violinplot(data=df['fetal_health'], ax = ax[2])
ax[2].set_xlabel('Fetal Health')
```

```
plt.show()
```



Will it be beneficial to scale the data?

Yes, scaling the data will be beneficial due to the fact that there is a significant variation in the data.

Is the data linearly separable?

Looking at the pairplots we can see that the data is not linearly separable using any two pairs of features. With this in mind, we cannot expect an accuracy close to 100% from a linear classifier.

Part II: Train/Test Split

Divide your dataset into training and testing subsets. Follow these steps to create the split:

1. **Divide the dataset into two data sets, each data set only contains samples of either class 0 or class 1:**
 - Create a DataFrame `df_0` containing all data with "fetal_health" equal to 0.
 - Create a DataFrame `df_1` containing all data with "fetal_health" equal to 1.
1. **Split into training and test set by randomly sampling entries from the data frames:**
 - Create a DataFrame `df_0_train` containing by sampling 75% of the entries from `df_0` (use the `sample` method of the data frame, fix the `random_state` to 42).
 - Create a DataFrame `df_1_train` using the same approach with `df_1`.
 - Create a DataFrame `df_0_test` containing the remaining entries of `df_0` (use `df_0.drop(df_0_train.index)` to drop all entries except the previously extracted ones).
 - Create a DataFrame `df_1_test` using the same approach with `df_1`.
1. **Merge the datasets split by classes back together:**
 - Create a DataFrame `df_train` containing all entries from `df_0_train` and `df_1_train`. (Hint: use the `concat` method you know from CA1)
 - Create a DataFrame `df_test` containing all entries from the two test sets.
1. **Create the following data frames from these splits:**

- `X_train`: Contains all columns of `df_train` except for the target feature `"fetal_health"`
 - `X_test`: Contains all columns of `df_test` except for the target feature `"fetal_health"`
 - `y_train`: Contains only the target feature `"fetal_health"` for all samples in the training set
 - `y_test`: Contains only the target feature `"fetal_health"` for all samples in the test set
1. **Check that your sets have the expected sizes/shape by printing number of rows and columns ("shape") of the data sets.**
 - (Sanity check: there should be 8 features, almost 1000 samples in the training set and slightly more than 300 samples in the test set.)
 1. **Explain the purpose of this slightly complicated procedure. Why did we first split into the two classes? Why did we then split into a training and a testing set?**
 2. **What is the share (in percent) of samples with class 0 label in test and training set, and in the initial data set?**

```
# Insert your code below
# =====
df_0 = df[df['fetal_health'] == 0]
df_1 = df[df['fetal_health'] == 1]

df_0_train = df_0.sample(frac=0.75, random_state=42)
df_1_train = df_1.sample(frac=0.75, random_state=42)

df_0_test = df_0.drop(df_0_train.index)
df_1_test = df_1.drop(df_1_train.index)

df_train = pd.concat([df_0_train, df_1_train])
df_test = pd.concat([df_0_test, df_1_test])

X_train = df_train.drop(['fetal_health'], axis=1)
X_test = df_test.drop(['fetal_health'], axis=1)
y_train = df_train['fetal_health']
y_test = df_test['fetal_health']

X_train.shape, X_test.shape, y_train.shape, y_test.shape

print(f'Precent of 0 label in the intial data set:
{round((len(df_0)/len(df))*100, 2)} %')
print(f'Precent of 0 label in the train data set:
{round(((len(df_train[df_train["fetal_health"]==0])/len(df_train))*100
),2)} %')
print(f'Precent of 0 label in the test data set:
{round(((len(df_test[df_test["fetal_health"]==0])/len(df_test))*100),2
)} %')
```

```
Precent of 0 label in the intial data set: 63.8 %  
Precent of 0 label in the train data set: 63.81 %  
Precent of 0 label in the test data set: 63.78 %
```

Explanation

Splitting the data into two classes ensures representation of both classes in the training and test sets, and it allows a balanced evaluation of model performance on unseen data. Further splitting the classes in train and test enables model training and evaluation and maintain a balance of the classes.

Convert data to numpy arrays and shuffle the training data

Many machine learning models (including those you will work with later in the assignment) will not accept DataFrames as input. Instead, they will only work if you pass numpy arrays containing the data. Here, we convert the DataFrames `X_train`, `X_test`, `y_train`, and `y_test` to numpy arrays `X_train`, `X_test`, `y_train`, and `y_test`.

Moreover we shuffle the training data. This is important because the training data is currently ordered by class. In Part IV, we use the first `n` samples from the training set to train the classifiers. If we did not shuffle the data, the classifiers would only be trained on samples of class 0.

Nothing to be done here, just execute the cell.

```
# convert to numpy arrays  
X_train = X_train.to_numpy()  
X_test = X_test.to_numpy()  
y_train = y_train.to_numpy()  
y_test = y_test.to_numpy()  
  
# shuffle training data  
np.random.seed(42) # for reproducibility  
shuffle_index = np.random.permutation(len(X_train)) # generate random  
indices  
X_train, y_train = X_train[shuffle_index], y_train[shuffle_index] #  
shuffle data by applying reordering with the random indices
```

Part III: Scaling the data

1. Standardize the training *and* test data so that each feature has a mean of 0 and a standard deviation of 1.
2. Check that the scaling was successful
 - by printing the mean and standard deviation of each feature in the scaled training set
 - by putting the scaled training set into a DataFrame and make a violin plot of the data

Hint: use the `axis` argument to calculate mean and standard deviation column-wise.

Important: Avoid data leakage!

More hints:

1. For each column, subtract the mean (μ) of each column from each value in the column
2. Divide the result by the standard deviation (σ) of the column

(You saw how to do both operations in the lecture. If you don't remember, you can look it up in Canvas files.)

Mathematically (in case this is useful for you), this transformation can be represented for each column as follows:

$$X_{\text{scaled}} = \frac{(X - \mu)}{\sigma}$$

where:

- (X_{scaled}) are the new, transformed column values (a column-vector)
- (X) is the original values
- (μ) is the mean of the column
- (σ) is the standard deviation of the column

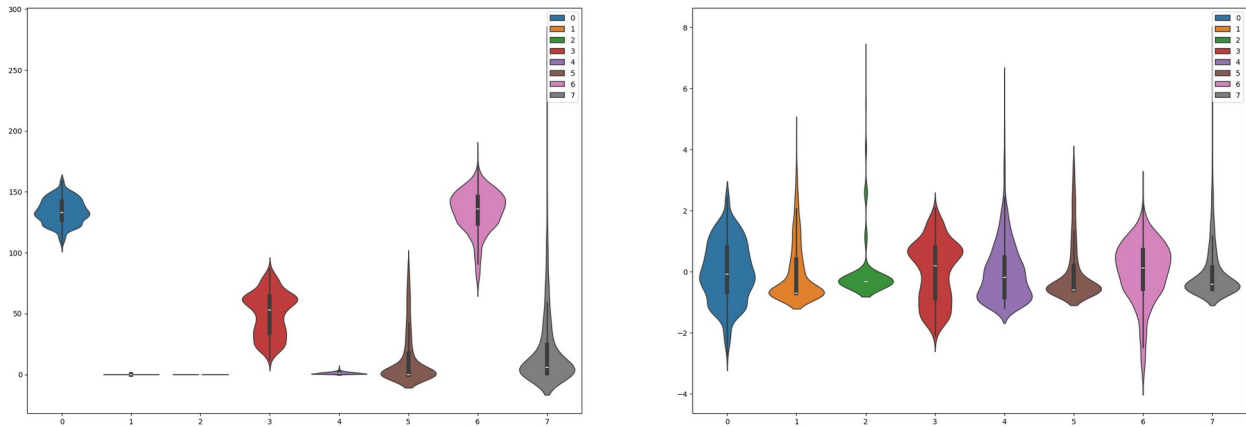
```
# Insert your code below
# =====
X_train_sc = (X_train - X_train.mean(axis=0)) / X_train.std(axis=0)
X_test_sc = (X_test - X_train.mean(axis=0)) / X_train.std(axis=0)

fig, axes = plt.subplots(1, 2, figsize=(30, 10))

sns.violinplot(X_train, legend='full', ax = axes[0])
sns.violinplot(X_train_sc, legend='full', ax = axes[1])

print(X_train_sc.std())
print(X_train_sc.mean())

1.0000000000000001
4.721031103680087e-16
```



Part IV: Training and evaluation with different dataset sizes and training times

Often, a larger dataset size will yield better model performance. (As we will learn later, this usually prevents overfitting and increases the generalization capability of the trained model.) However, collecting data is usually rather expensive.

In this part of the exercise, you will investigate

- how the model performance changes with varying dataset size
- how the model performance changes with varying numbers of epochs/iterations of the optimizer/solver (increasing training time).

For this task (Part IV), use the `Adaline`, `Perceptron`, and `LogisticRegression` classifier from the `mlxtend` library. All use the gradient descent (GD) algorithm for training.

Important: Use a learning rate of `1e-4` (`0.0001`) for all classifiers, and use the argument `minibatches=1` when initializing `Adaline` and `LogisticRegression` classifier (this will make sure it uses GD). For all three classifiers, pass `random_seed=42` when initializing the classifier to ensure reproducibility of the results.

Model training

Train the model models using progressively larger subsets of your dataset, specifically: first 50 rows, first 100 rows, first 150 rows, ..., first 650 rows, first 700 rows (in total 14 different variants).

For each number of rows train the model with progressively larger number of epochs: 2, 7, 12, 17, ..., 87, 92, 97 (in total 20 different model variants).

The resulting $14 \times 20 = 280$ models obtained from the different combinations of subsets and number of epochs. An output of the training process could look like this:

```
Model (1) Train a model with first 50 rows of data for 2 epochs
Model (2) Train a model with first 50 rows of data for 7 epochs
Model (3) Train a model with first 50 rows of data for 12 epochs
```



```

...
Model (21) Train a model with first 100 rows of data for 2 epochs
Model (22) Train a model with first 100 rows of data for 7 epochs
...
Model (279) Train a model with first 700 rows of data for 92 epochs
Model (280) Train a model with first 700 rows of data for 97 epochs

```

Model evaluation

For each of the 280 models, calculate the **accuracy on the test set** (do **not** use the score method but compute accuracy yourself). Store the results in the provided 2D numpy array (it has 14 rows and 20 columns). The rows of the array correspond to the different dataset sizes, and the columns correspond to the different numbers of epochs.

Tasks

1. Train the 280 Adaline classifiers as mentioned above and calculate the accuracy for each of the 280 variants.
2. Generalize your code so that is doing the same procedure for all three classifiers: **Perceptron**, **Adaline**, and **LogisticRegression** after each other. Store the result for all classifiers. You can for example use an array of shape $3 \times 14 \times 20$ to store the accuracies of the three classifiers.

Note that executing the cells will take some time (but on most systems it should not be more than 5 minutes).

```

# Train and evaluate all model variants
# Insert your code below
# =====
rows = list(range(50,750,50))
epochs = list(range(2,102,5))
models = [Perceptron, Adaline, LogisticRegression]

storages = np.zeros([3,14,20])

def accuracy_score(x, y):
    correct_predictions = (x == y).sum()
    total_predictions = len(y)
    return correct_predictions / total_predictions

for k, model in enumerate(models):
    for i, row in enumerate(rows):
        for j, epoch in enumerate(epochs):
            if model == Perceptron:
                mod = model(epochs=epoch, eta=0.0001, random_seed=42)
                mod.fit(X_train_sc[:row], y_train[:row])
                y_pred = mod.predict(X_test_sc)
                accuracy = accuracy_score(y_test, y_pred)
                print(accuracy)
                storages[k][i][j] = accuracy

```

```

        else:
            mod = model(epochs=epoch, eta=0.0001, random_seed=42,
minibatches=1)
            mod.fit(X_train_sc[:row], y_train[:row])
            y_pred = mod.predict(X_test_sc)
            accuracy = accuracy_score(y_test, y_pred)
            print(accuracy)
            storages[k][i][j] = accuracy

```

```

0.8111455108359134
0.8637770897832817
0.8637770897832817
0.8668730650154799
0.8823529411764706
0.8792569659442725
0.8947368421052632
0.9009287925696594
0.9009287925696594
0.891640866873065
0.8947368421052632
0.8885448916408669
0.8699690402476781
0.8885448916408669
0.8885448916408669
0.8699690402476781
0.8761609907120743
0.8513931888544891
0.891640866873065
0.8854489164086687
0.826625386996904
0.8792569659442725
0.8792569659442725
0.8823529411764706
0.8792569659442725
0.891640866873065
0.8606811145510835
0.826625386996904
0.8699690402476781
0.8668730650154799
0.7647058823529411
0.8699690402476781
0.8792569659442725
0.8606811145510835
0.8792569659442725
0.8637770897832817
0.8761609907120743
0.8730650154798761
0.8854489164086687
0.8699690402476781
0.8421052631578947

```

0.8668730650154799
0.8668730650154799
0.826625386996904
0.8452012383900929
0.8328173374613003
0.8730650154798761
0.8575851393188855
0.8792569659442725
0.6718266253869969
0.8421052631578947
0.8668730650154799
0.8297213622291022
0.8390092879256966
0.8792569659442725
0.8544891640866873
0.8885448916408669
0.8668730650154799
0.8699690402476781
0.8637770897832817
0.8668730650154799
0.891640866873065
0.8854489164086687
0.8359133126934984
0.9040247678018576
0.8823529411764706
0.8792569659442725
0.8823529411764706
0.8544891640866873
0.8297213622291022
0.8328173374613003
0.8235294117647058
0.9009287925696594
0.8730650154798761
0.8730650154798761
0.7925696594427245
0.8513931888544891
0.8978328173374613
0.9040247678018576
0.891640866873065
0.8699690402476781
0.8575851393188855
0.8978328173374613
0.8606811145510835
0.8699690402476781
0.8854489164086687
0.8452012383900929
0.8452012383900929
0.8699690402476781
0.8390092879256966

0.8390092879256966
0.8978328173374613
0.8544891640866873
0.8544891640866873
0.8823529411764706
0.891640866873065
0.891640866873065
0.891640866873065
0.891640866873065
0.7894736842105263
0.8544891640866873
0.8452012383900929
0.8544891640866873
0.8823529411764706
0.8637770897832817
0.8606811145510835
0.8885448916408669
0.7894736842105263
0.8885448916408669
0.8359133126934984
0.6130030959752322
0.8823529411764706
0.8452012383900929
0.8730650154798761
0.8823529411764706
0.8792569659442725
0.8947368421052632
0.8390092879256966
0.8699690402476781
0.826625386996904
0.8637770897832817
0.8885448916408669
0.8575851393188855
0.8761609907120743
0.8328173374613003
0.8885448916408669
0.8823529411764706
0.8699690402476781
0.8544891640866873
0.8699690402476781
0.8761609907120743
0.8606811145510835
0.8823529411764706
0.8513931888544891
0.8668730650154799
0.891640866873065
0.8854489164086687
0.8823529411764706
0.8823529411764706

0.8792569659442725
0.8637770897832817
0.8792569659442725
0.8885448916408669
0.8668730650154799
0.8235294117647058
0.9009287925696594
0.8792569659442725
0.8730650154798761
0.8575851393188855
0.7337461300309598
0.8328173374613003
0.8699690402476781
0.8885448916408669
0.7430340557275542
0.8854489164086687
0.7368421052631579
0.8823529411764706
0.8947368421052632
0.8885448916408669
0.8575851393188855
0.8792569659442725
0.8978328173374613
0.8761609907120743
0.7801857585139319
0.8699690402476781
0.8761609907120743
0.7925696594427245
0.8823529411764706
0.8823529411764706
0.8359133126934984
0.8730650154798761
0.8792569659442725
0.8730650154798761
0.848297213622291
0.8854489164086687
0.8792569659442725
0.8328173374613003
0.8575851393188855
0.8544891640866873
0.6006191950464397
0.8668730650154799
0.8699690402476781
0.8730650154798761
0.8854489164086687
0.8359133126934984
0.7894736842105263
0.8947368421052632
0.804953560371517

0.7213622291021672
0.8761609907120743
0.7523219814241486
0.8452012383900929
0.8544891640866873
0.8699690402476781
0.8575851393188855
0.8885448916408669
0.8792569659442725
0.8235294117647058
0.8885448916408669
0.8761609907120743
0.8421052631578947
0.8885448916408669
0.9102167182662538
0.7987616099071208
0.7678018575851393
0.8854489164086687
0.8792569659442725
0.8792569659442725
0.9071207430340558
0.8699690402476781
0.8885448916408669
0.6718266253869969
0.8297213622291022
0.8359133126934984
0.8823529411764706
0.8761609907120743
0.8823529411764706
0.8885448916408669
0.8854489164086687
0.8359133126934984
0.8792569659442725
0.8761609907120743
0.9009287925696594
0.891640866873065
0.8823529411764706
0.891640866873065
0.8730650154798761
0.8328173374613003
0.7987616099071208
0.848297213622291
0.8359133126934984
0.8885448916408669
0.8885448916408669
0.8854489164086687
0.891640866873065
0.9009287925696594
0.8699690402476781

0.891640866873065
0.7213622291021672
0.848297213622291
0.8173374613003096
0.8792569659442725
0.9009287925696594
0.8235294117647058
0.6718266253869969
0.8885448916408669
0.7275541795665634
0.8204334365325078
0.8204334365325078
0.8606811145510835
0.7678018575851393
0.8606811145510835
0.8297213622291022
0.8637770897832817
0.8390092879256966
0.8390092879256966
0.7863777089783281
0.8637770897832817
0.8080495356037152
0.8018575851393189
0.8421052631578947
0.8792569659442725
0.739938080495356
0.628482972136223
0.8452012383900929
0.804953560371517
0.8575851393188855
0.8297213622291022
0.8668730650154799
0.8637770897832817
0.8606811145510835
0.8885448916408669
0.8730650154798761
0.8544891640866873
0.6594427244582043
0.8018575851393189
0.7770897832817337
0.8668730650154799
0.7368421052631579
0.8575851393188855
0.8018575851393189
0.8513931888544891
0.8668730650154799
0.8637770897832817
0.8637770897832817
0.8606811145510835

0.8606811145510835
0.8575851393188855
0.8575851393188855
0.8575851393188855
0.8606811145510835
0.8606811145510835
0.8637770897832817
0.8637770897832817
0.8637770897832817
0.8668730650154799
0.8699690402476781
0.8668730650154799
0.8668730650154799
0.8699690402476781
0.826625386996904
0.8730650154798761
0.8699690402476781
0.8699690402476781
0.8761609907120743
0.8761609907120743
0.8792569659442725
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8792569659442725
0.8792569659442725
0.8792569659442725
0.8792569659442725
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8452012383900929
0.8699690402476781
0.8668730650154799
0.8730650154798761
0.8792569659442725
0.8823529411764706
0.8792569659442725
0.8792569659442725
0.8792569659442725
0.8792569659442725
0.8761609907120743
0.8761609907120743
0.8730650154798761
0.8730650154798761
0.8730650154798761

0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8452012383900929
0.8668730650154799
0.8668730650154799
0.8699690402476781
0.8699690402476781
0.8668730650154799
0.8730650154798761
0.8730650154798761
0.8792569659442725
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8792569659442725
0.8792569659442725
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8544891640866873
0.8606811145510835
0.8668730650154799
0.8668730650154799
0.8699690402476781
0.8730650154798761
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8699690402476781
0.8699690402476781
0.8637770897832817
0.8699690402476781
0.8637770897832817
0.8699690402476781

0.8792569659442725
0.8761609907120743
0.8792569659442725
0.8761609907120743
0.8761609907120743
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8699690402476781
0.8637770897832817
0.8730650154798761
0.8668730650154799
0.8699690402476781
0.8730650154798761
0.8730650154798761
0.8761609907120743
0.8761609907120743
0.8730650154798761
0.8761609907120743
0.8792569659442725
0.8792569659442725
0.8792569659442725
0.8792569659442725
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8637770897832817
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8699690402476781
0.8761609907120743
0.8792569659442725
0.8761609907120743
0.8792569659442725
0.8761609907120743
0.8730650154798761
0.8730650154798761
0.8730650154798761

0.8699690402476781
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8637770897832817
0.8761609907120743
0.8699690402476781
0.8761609907120743
0.8792569659442725
0.8730650154798761
0.8730650154798761
0.8761609907120743
0.8792569659442725
0.8823529411764706
0.8823529411764706
0.8854489164086687
0.8854489164086687
0.8854489164086687
0.8854489164086687
0.8854489164086687
0.8885448916408669
0.8885448916408669
0.8885448916408669
0.8885448916408669
0.8885448916408669
0.8637770897832817
0.8668730650154799
0.8668730650154799
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8792569659442725
0.8854489164086687
0.8854489164086687
0.8854489164086687
0.8854489164086687
0.8854489164086687
0.8885448916408669
0.8885448916408669
0.891640866873065
0.8885448916408669
0.8885448916408669
0.8885448916408669
0.8885448916408669
0.891640866873065
0.8637770897832817
0.8699690402476781

0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8792569659442725
0.8823529411764706
0.8854489164086687
0.8885448916408669
0.8885448916408669
0.891640866873065
0.8885448916408669
0.8885448916408669
0.8885448916408669
0.8885448916408669
0.8854489164086687
0.8854489164086687
0.8854489164086687
0.8854489164086687
0.8637770897832817
0.8730650154798761
0.8761609907120743
0.8792569659442725
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8792569659442725
0.8854489164086687
0.8885448916408669
0.8885448916408669
0.891640866873065
0.8885448916408669
0.8885448916408669
0.891640866873065
0.8854489164086687
0.8854489164086687
0.8854489164086687
0.8854489164086687
0.8854489164086687
0.8854489164086687
0.8637770897832817
0.8699690402476781
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8730650154798761
0.8761609907120743
0.8823529411764706
0.8823529411764706
0.8854489164086687
0.8854489164086687
0.8885448916408669

0.8885448916408669
0.8854489164086687
0.8885448916408669
0.8885448916408669
0.8885448916408669
0.8885448916408669
0.8885448916408669
0.8854489164086687
0.8637770897832817
0.8699690402476781
0.8761609907120743
0.8730650154798761
0.8761609907120743
0.8792569659442725
0.8792569659442725
0.8823529411764706
0.8823529411764706
0.8854489164086687
0.8885448916408669
0.8885448916408669
0.8885448916408669
0.8854489164086687
0.8885448916408669
0.8885448916408669
0.8885448916408669
0.8854489164086687
0.8885448916408669
0.8885448916408669
0.8885448916408669
0.8885448916408669
0.8885448916408669
0.8885448916408669
0.8885448916408669
0.8885448916408669
0.8885448916408669
0.7832817337461301
0.8235294117647058
0.8452012383900929
0.8513931888544891
0.8606811145510835
0.8668730650154799
0.8668730650154799
0.8637770897832817
0.8637770897832817
0.8606811145510835
0.8637770897832817
0.8575851393188855
0.8544891640866873
0.8544891640866873
0.8544891640866873
0.8544891640866873
0.8575851393188855
0.8544891640866873
0.8575851393188855
0.8575851393188855
0.8575851393188855
0.8018575851393189

0.8575851393188855
0.8792569659442725
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8730650154798761
0.8730650154798761
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8792569659442725
0.8792569659442725
0.8792569659442725
0.8792569659442725
0.8080495356037152
0.8637770897832817
0.8668730650154799
0.8699690402476781
0.8730650154798761
0.8730650154798761
0.8699690402476781
0.8699690402476781
0.8761609907120743
0.8730650154798761
0.8761609907120743
0.8792569659442725
0.8792569659442725
0.8792569659442725
0.8792569659442725
0.8823529411764706
0.8823529411764706
0.8823529411764706
0.8823529411764706
0.8823529411764706
0.8823529411764706
0.8142414860681114
0.8606811145510835
0.8668730650154799
0.8668730650154799
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8699690402476781

0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8668730650154799
0.8668730650154799
0.8668730650154799
0.8204334365325078
0.8730650154798761
0.8637770897832817
0.8606811145510835
0.8668730650154799
0.8668730650154799
0.8668730650154799
0.8668730650154799
0.8668730650154799
0.8668730650154799
0.8668730650154799
0.8668730650154799
0.8668730650154799
0.8668730650154799
0.8637770897832817
0.8637770897832817
0.8668730650154799
0.8668730650154799
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8452012383900929
0.8637770897832817
0.8668730650154799
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8668730650154799
0.8668730650154799
0.8668730650154799
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8699690402476781
0.8699690402476781
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8761609907120743
0.8792569659442725

0.8792569659442725
0.8452012383900929
0.8637770897832817
0.8668730650154799
0.8637770897832817
0.8699690402476781
0.8730650154798761
0.8699690402476781
0.8699690402476781
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8699690402476781
0.8699690402476781
0.8668730650154799
0.8668730650154799
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8606811145510835
0.8668730650154799
0.8668730650154799
0.8792569659442725
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8730650154798761
0.8668730650154799
0.8637770897832817
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8730650154798761
0.8730650154798761
0.8699690402476781

0.8730650154798761
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8730650154798761
0.8761609907120743
0.8761609907120743
0.8792569659442725
0.8823529411764706
0.8637770897832817
0.8637770897832817
0.8730650154798761
0.8730650154798761
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8668730650154799
0.8668730650154799
0.8668730650154799
0.8668730650154799
0.8699690402476781
0.8699690402476781
0.8730650154798761
0.8730650154798761
0.8792569659442725
0.8761609907120743
0.8761609907120743
0.8761609907120743
0.8823529411764706
0.8637770897832817
0.8637770897832817
0.8761609907120743
0.8730650154798761
0.8699690402476781
0.8730650154798761
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8730650154798761
0.8761609907120743
0.8823529411764706
0.8792569659442725
0.8792569659442725
0.8792569659442725
0.8823529411764706

0.8761609907120743
0.8761609907120743
0.8730650154798761
0.8699690402476781
0.8637770897832817
0.8761609907120743
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8730650154798761
0.8792569659442725
0.8823529411764706
0.8792569659442725
0.8792569659442725
0.8792569659442725
0.8823529411764706
0.8792569659442725
0.8792569659442725
0.8637770897832817
0.8668730650154799
0.8730650154798761
0.8699690402476781
0.8699690402476781
0.8699690402476781
0.8668730650154799
0.8668730650154799
0.8699690402476781
0.8730650154798761
0.8730650154798761
0.8730650154798761
0.8761609907120743
0.8792569659442725
0.8792569659442725
0.8761609907120743
0.8761609907120743
0.8792569659442725
0.8792569659442725
0.8761609907120743
0.8637770897832817
0.8637770897832817
0.8699690402476781
0.8730650154798761
0.8668730650154799
0.8699690402476781

```
0.8668730650154799
0.8668730650154799
0.8699690402476781
0.8730650154798761
0.8730650154798761
0.8761609907120743
0.8792569659442725
0.8792569659442725
0.8792569659442725
0.8761609907120743
0.8761609907120743
0.8792569659442725
0.8792569659442725
0.8761609907120743
```

Performance visualization

Plot the performance measure for all classifiers (accuracy on the test set; use the result array from above) of all the 280 variants for each classifier in a total of three heatmaps using, for example `seaborn` or `matplotlib` directly.

The color should represent the accuracy on the test set, and the x and y axes should represent the number of epochs and the dataset size, respectively. Which one is x and which one is y is up to you to decide. Look in the example output at the top of the assignment for inspiration for how the plot could look like and how it could be labeled nicely. (But use the correct numbers corresponding to your dataset sizes and number of epochs.)

```
# Insert your code below
# =====
perceptron_accuracy = storages[0]
adaline_accuracy = storages[1]
reg_line_accuracy = storages[2]

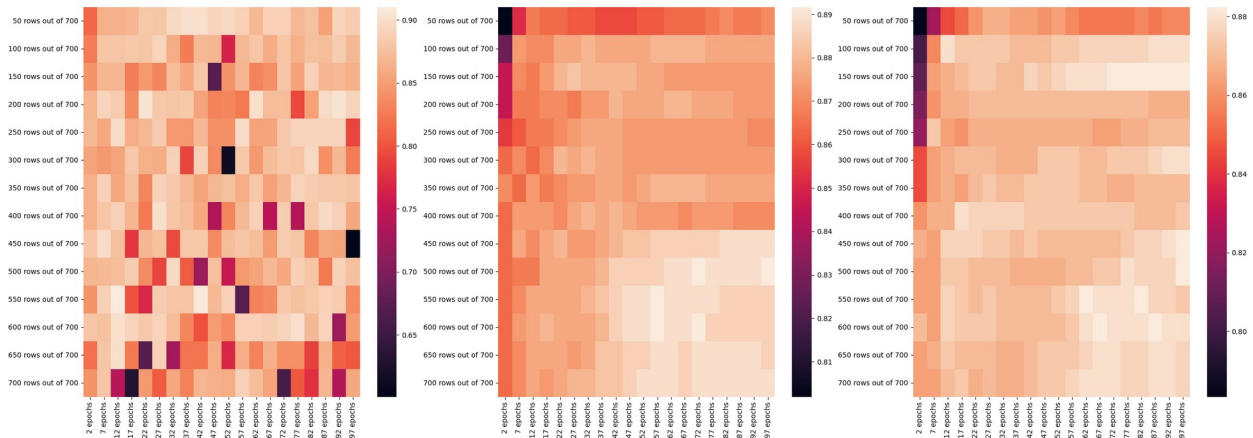
fig, axes = plt.subplots(1, 3, figsize=(30, 10))

x_label = [f'{i} epochs' for i in epochs]
y_label = [f'{row} rows out of 700' for row in rows]
sns.heatmap(perceptron_accuracy, xticklabels=x_label, yticklabels =
y_label, ax=axes[0])

x_label = [f'{i} epochs' for i in epochs]
y_label = [f'{row} rows out of 700' for row in rows]
sns.heatmap(adaline_accuracy, xticklabels=x_label, yticklabels =
y_label, ax=axes[1])

x_label = [f'{i} epochs' for i in epochs]
y_label = [f'{row} rows out of 700' for row in rows]
sns.heatmap(reg_line_accuracy, xticklabels=x_label, yticklabels =
y_label, ax=axes[2])
```

<Axes: >



Part V: Some more plotting

For the following cell to execute you need to have the variable `X_test_scaled` with all samples of the test set and the variable `y_test` with the corresponding labels. Complete at least up until Part III. Executing the cell will plot something.

1. Add code comments explaining what the lines are doing
2. What is the purpose of the plot?
3. Describe all components of the subplot and then comment in general on the entire plot. What does it show? What does it not show?

```
# Train and a logistic regression model with 300 epochs and learning
rate 0.0001
X_test_scaled = X_test_sc # rename variable
clf = LogisticRegression(eta = 0.0001, epochs = 300, minibatches=1,
random_seed=42)# Create a logistic regression classifier with
specified parameters
clf.fit(X_test_scaled, y_test)# Fit the logistic regression model to
the scaled test data and test labels

# Create a 8x8 subplot grid for plotting
fig, axes = plt.subplots(8, 8, figsize=(30, 30))
for i in range(0, 8):
    for j in range(0, 8):
        feature_1 = i
        feature_2 = j
        ax = axes[i, j] # position for the subplot

        # Set x and y labels for the subplot
        ax.set_xlabel(f"Feature {feature_1}")
        ax.set_ylabel(f"Feature {feature_2}")
```

```

# Compute minimum and maximum values for scaling
mins = X_test_scaled.min(axis=0)
maxs = X_test_scaled.max(axis=0)

# Generate grid of points for the two features
x0 = np.linspace(mins[feature_1], maxs[feature_1], 100)
x1 = np.linspace(mins[feature_2], maxs[feature_2], 100)
X0, X1 = np.meshgrid(x0, x1) # Create meshgrid for contour
plot
X_two_features = np.c_[X0.ravel(), X1.ravel()] # Combine
features
X_plot = np.zeros(shape=(X_two_features.shape[0],
X_test_scaled.shape[1]))

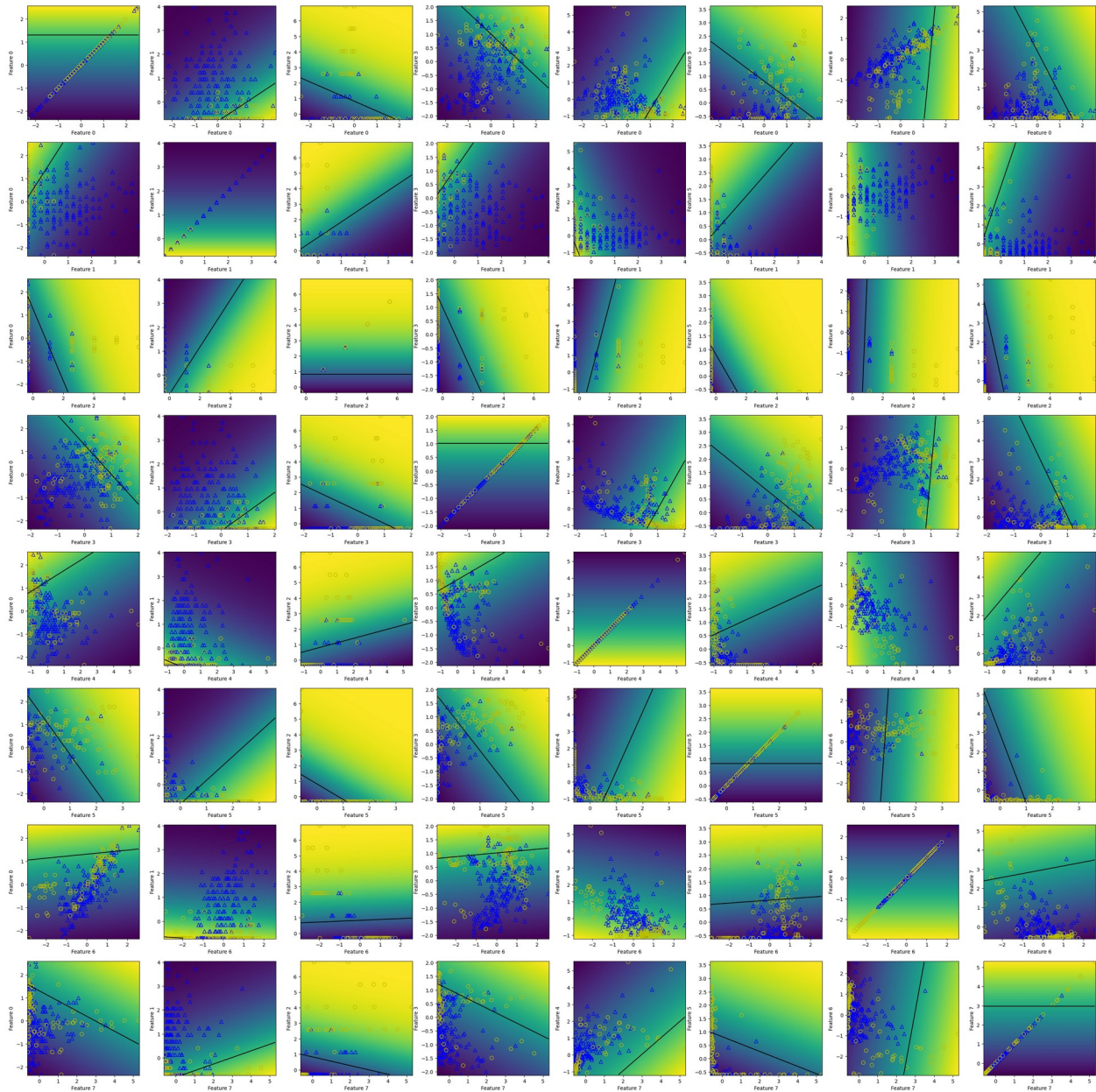
# Set values for the two features
X_plot[:, feature_1] = X_two_features[:, 0]
X_plot[:, feature_2] = X_two_features[:, 1]

# Predict probabilities for the combined features
y_pred = clf.predict_proba(X_plot)
Z = y_pred.reshape(X0.shape)

# Plot the probabilities as a filled contour plot
ax.pcolor(X0, X1, Z)
# Add contour lines at the decision boundary (probability =
0.5)
ax.contour(X0, X1, Z, levels=[0.5], colors='k')
# Plot points of class 0 and class 1
ax.scatter(X_test_scaled[y_test == 0, feature_1],
X_test_scaled[y_test == 0, feature_2], color="b", marker="^", s=50,
facecolors="none")
ax.scatter(X_test_scaled[y_test == 1, feature_1],
X_test_scaled[y_test == 1, feature_2], color="y", marker="o", s=50,
facecolors="none")

fig.tight_layout()
plt.show()

```



Part VI: Additional discussion

Part I:

1. What kind of plots did you use to visualize the raw data, and why did you choose these types of plots?

Part II:

1. What happens if we don't shuffle the training data before training the classifiers like in Part IV?
2. How could you do the same train/test split (Point 1.-4.) using scikit-learn?

Part IV:

1. How does increasing the dataset size affect the performance of the logistic regression model? Provide a summary of your findings.
2. Describe the relationship between the number of epochs and model accuracy
3. Which classifier is much slower to train and why do you think that is?
4. One classifier shows strong fluctuations in accuracy for different dataset sizes and number of epochs. Which one is it and why do you think this happens?

Answers Part VI

Part I:

1. I used seaborn pairplot to visualize the raw data to see if the data was linearly separable. I also used the seaborn violinplot to describe the distribution of the data.

Part II:

1. If we don't shuffle the training data before training the classifiers, you get an unbalanced and bad performing model. With not shuffling the data we get the risk of introducing bias into the model, and the model might learn "wrong", e.g., specific patterns/sequences specific to the order of the data, related to how it was collected. This will lead to a poorly performing model on unseen data, due to the fact that it has not generalized on diverse sample data. Not shuffling the data may also lead to the model prioritizing to learn more from some subsets of the data than others, which lead to an unbalanced representation of the classes.
2. You could do the same train/test split utilizing shuffle and stratify attributes from the scikit-learn package.

Part IV:

1. By increasing the size of the dataset, the accuracy of the logistic regression model will be more accurate. A higher number of epochs will also increase the model's performance.
2. The relationship between number of epochs and accuracy in the perceptron model looks quite random and shows not so consistent patterns. In the Logistic regression and Adaline model there is a more clear relationship between the accuracy and number of epochs. The accuracy gets better with a higher number of epochs. I believe these models perform better with more data and training iterations.
3. The perceptron model is much more slower to train because it requires more time to converge to a solution, distinctly when the data is not linearly separable. The perceptron algorithm may require more iterations to adjust weights, and therefore is slower than the adaline and log. reg., which converge to a solution for non-linearly separable data.
4. The perceptron model shows strong fluctuations in accuracy for different dataset sizes and number of epochs. I believe this is because it struggles to converge to the

optimal weights, especially when the data is not linearly separable. This may make it hard for the model to find a decision boundary that separates the classes, and therefore lead to an inconsistent performance. With this in mind, the perceptron models accuracy vary depending on the datasets characteristics and number of iterations in the training.